

VIBECODING COURSE

AI 코드 품질 심화 실전 과정

Gemini CLI × Cursor AI

프롬프트 엔지니어링 + AI 코딩 도구 완전 정복 | VSCode C++ 중심 & Java 병행 개발자 과정

🕒 8시간

📄 100슬라이드

◆ Gemini CLI

📄 Cursor AI

⚙️ C++17

☕ Java 17+

학습 로드맵

각 시간마다 실습 프로젝트 완성 → 포트폴리오 8개 구축

1시간

프롬프트 엔지니어링 기초

CLEAR 프레임워크 · PRD 작성 · 역할 부여 · 반복 개선

2시간

Gemini CLI 설치 & 기초 실습

npm 설치 · 손글씨 인식 · GitHub · 외부 서비스 연동

3시간

Gemini CLI 고급 — Todo 앱

PRD→5단계 개발 · C++/Java 모델 · 커스텀 명령어

4시간

Cursor AI 설치 & 코드 품질

.cursorrules · Tab·Ctrl+K·Ctrl+L · C++/Java 리팩토링

5시간

Cursor AI 고급 — 에이전트

멀티파일 · OpenRouter API · 냉장고 앱 · AI 다이어리

6시간

실전 프로젝트 1 — 퀴즈 게임

3단계 퀴즈 게임 · 점수 시스템 · 리더보드

7시간

MCP 입문 — 도구 연결

Notion · Sequential Thinking · Playwright · GitHub MCP

8시간

MCP 고급 & 총정리

Supabase DB · Context7 · 워크플로우 · 포트폴리오 완성

1번째 시간

프롬프트 엔지니어링 기초

AI를 200% 활용하는 효과적인 프롬프트 설계 원칙



생성형 AI란 무엇인가?

AI Literacy = AI 이해 + AI 활용 + AI 판단 능력 — 2026년은 코딩보다 "의도를 정확히 전달하는" 역량이 핵심

생성형 AI (Generative AI)

프롬프트에 대응해 텍스트·이미지·코드 등 콘텐츠를 생성하는 AI 시스템
입력 데이터의 패턴·구조를 학습 → 유사 특성의 새 데이터 생성
대표 모델: ChatGPT, Claude, Gemini, Midjourney, DALL-E

LLM (Large Language Model)

Transformer 기반 딥러닝 모델
방대한 텍스트 데이터로 사전 학습(Pre-training)
자연어 이해·생성·번역·코딩 등 다목적 활용
핵심: Attention 메커니즘 → 문맥 이해

Prompt → Model → Output 핵심 흐름

① Prompt

사용자 입력
자연어 지시



② Tokenize

텍스트 → Token
숫자 벡터 변환



③ Attention

문맥 이해
중요 단어 집중



④ Generate

다음 토큰 예측
답변 완성

AI와 기계학습의 발전 과정 (트랜스포머 이전)



핵심 문제: RNN/LSTM은 문장이 길어질수록 초반 정보를 "압축-손실"함 → Thought Vector 병목 → Attention으로 해결

트랜스포머 아키텍처의 혁명

Attention 메커니즘

- Encoder가 입력 전체를 하나의 벡터로 압축하는 병목을 제거
- Decoder가 매 스텝마다 입력 전체를 참조 → 필요한 부분에 집중

"Are you free tomorrow?" → 번역 시 각 단어가 어디에 주목하는지 계산

나는 지금 '시간'까지 말했고, 다음 단어를 만들려면
입력 문장에서 뭘 봐야 할까?"라는 질문 벡터時

- Q(Query) · K(Key) · V(Value)의 내적으로 중요도 계산
- 결과: Are: 0.00 you: 0.01 free: 0.74 tomorrow: 0.24 ?: 0.01
→ context 벡터는 74%는 "free"의 의미 + 24%는
"tomorrow"의 의미를 담고 있음

Transformer가 해결한 것

고전 Seq2Seq	Transformer
고정 벡터 병목	병목 없음
장거리 의존성 X	어디서나 참조 
순차 처리(느림)	병렬 처리(빠름)
LSTM 기반	Self-Attention

2017년 이후 GPT·BERT·LLaMA·Claude 등 모든 SOTA 모델은
Transformer 기반

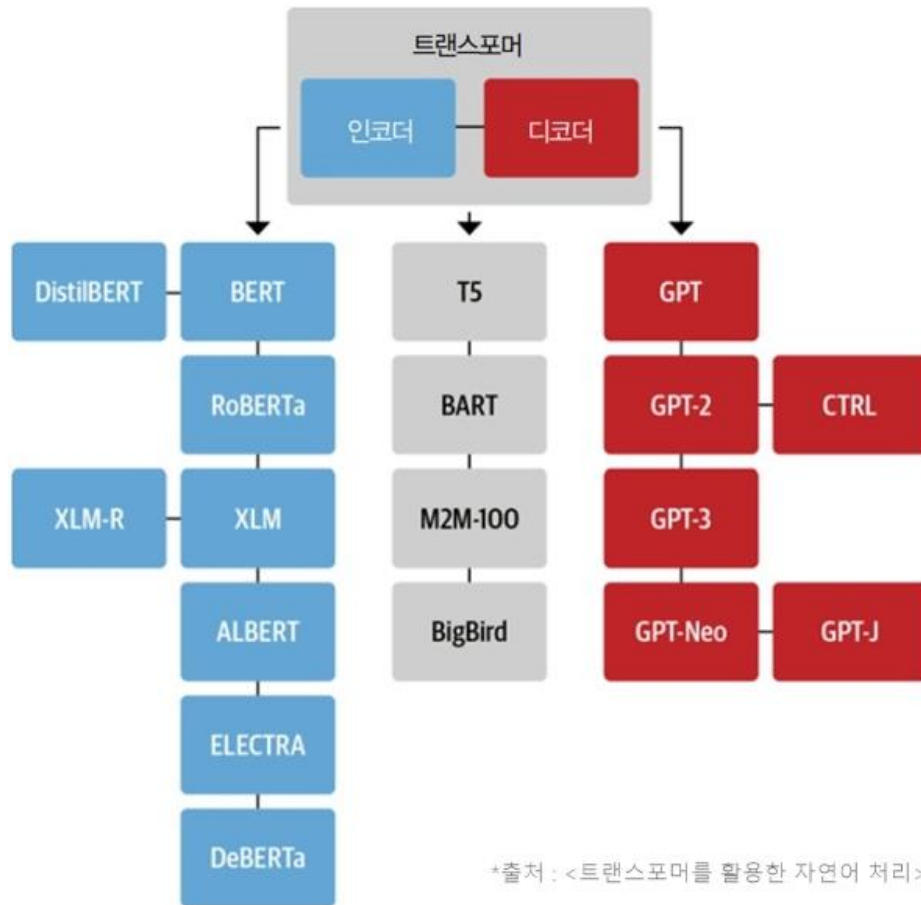
BERT 와 GPT



전략적 감독과 판단



책임



*출처 : <트랜스포머를 활용한 자연어 처리>, 한빛미디어, 2022

Transformer · Attention · RLHF — LLM 작동 원리

⚡ Attention 메커니즘 — LLM의 핵심

Query

현재 단어가 "무엇을 보아야 하나?" 라는 질문 벡터

Key

입력 문장 각 단어의 "식별 태그"

Value

실제 의미 정보 — 가중 평균으로 Context 생성

Softmax

유사도 점수를 확률 분포로 정규화

Context

"free 74% + tomorrow 24%" 처럼 중요 단어 집중

→ 2017년 이후 모든 SOTA(GPT·Claude·Llama)는 Attention 기반 Transformer 사용

🎯 RLHF — ChatGPT를 만든 기술

①

사전 학습

GPT-3 등 대규모 텍스트
비지도 학습으로 언어 능력 습득

②

초기 응답 생성

프롬프트 → 여러 응답 생성
인간 평가자가 품질 평가

③

보상 모델 학습

평가 데이터로 Reward Model 훈련
"좋은 응답"을 예측하는 모델

④

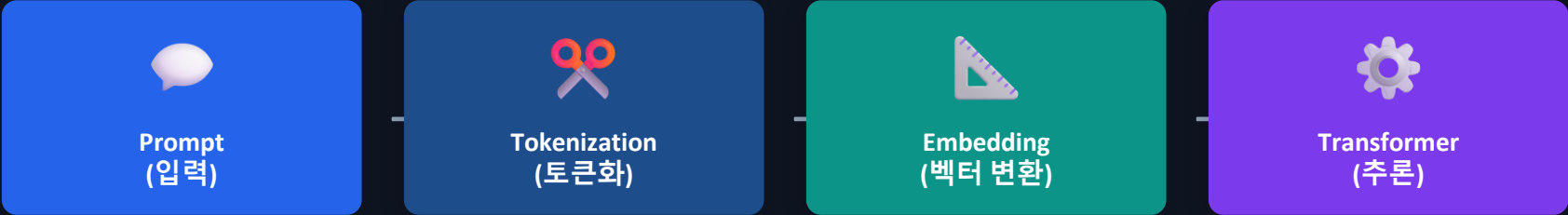
강화 학습 적용

PPO 알고리즘으로 LLM 파인튜닝
높은 보상을 받는 응답 강화

Closed vs Open Source LLM

Closed: GPT-4o, Claude 3.5 → API 방식, 높은 성능
Open : Llama 3.1, Qwen 2.5 → 로컬 실행, 보안/비용
↑

Prompt → Model → Output 흐름



SOTA는 State-of-the-Art의 약자로, AI/ML 분야에서 특정 작업(예: 텍스트 생성, 이미지 인식, 번역 등)에 대해 현재 최고 성능을 내는 모델

분야	SOTA 모델 예시	주요 강점	벤치마크 예시 (2025년 최고 점수)
LLM (NLP)	GPT-4o (OpenAI)	실시간 대화, 멀티모달 (텍스트+이미지+음성), 함수 호출 지원	MMLU: 88%+, HumanEval: 90%+
LLM (오픈소스)	Llama 3.1 (Meta), Qwen 2.5 (Alibaba)	오픈소스라 fine-tuning 쉽고, 컨텍스트 윈도우 128K+ 토큰 지원	MT-Bench: 8.5/10, GSM8K: 95%
멀티모달	Gemini 1.5 (Google), DALL-E 3 (OpenAI)	이미지/비디오 생성+이해, MMMU 같은 시험에서 인간 수준	MMMU: 65%+, VQA: 85%
비전 (CV)	DINOv3 (Meta), Segment Anything (SAM, Meta)	자가 지도 학습으로 이미지 세그멘테이션, 1B+ 마스크 데이터셋	ImageNet: 90%+, COCO: 60% mAP
추론/에이전트	o3-mini (OpenAI), Claude 3.5 (Anthropic)	구조화된 출력, 도구 통합 (tool calling), 비용 효율적	ARC: 75%+, GPQA: 50%+
기타 (RAG/에이전트)	RAG-Anything, Agent0	멀티모달 지식 검색, 자가 진화 에이전트 (인간 데이터 없이 학습)	복합 벤치마크: 20%+ 향상

프롬프트란? — AI와 대화하는 언어

프롬프트 엔지니어링

프롬프트(Prompt)란?

인공지능에 지시하는 명령어 또는 AI와의 대화를 위한 질문, 지시, 요청.

더 넓게는 AI가 의도를 이해하고 적절한 응답을 생성하도록 안내하는 모든 형태의 입력.

마치 음악가가 악기를 다루듯,
작가가 펜을 다루듯,
프롬프트 엔지니어는 언어를 통해 AI의 잠재력을 이끌어냄.

좋은 프롬프트의 효과

왜 중요한가?

같은 AI라도 프롬프트를 어떻게 작성하느냐에 따라 전혀 다른 결과 제공.

나쁜 예:

'코드 짜줘'

→ AI: '어떤 코드를 원하시나요?'

좋은 예 (P-C-T-F):

'너는 Python 전문가 [Persona].

사용자 인증 시스템 개발 중 [Context].

JWT 기반 로그인 함수 작성 [Task].

타입 힌트 포함, 주석 한국어 [Format].'

→ 정확한 코드 즉시 생성

프롬프트란? — AI와 대화하는 언어

나쁜 프롬프트 vs 좋은 프롬프트 비교

✖ 나쁜 프롬프트

프롬프트

홈페이지 만들어줘

문제점:

- ▶ 목적·대상·스타일 정보 없음
- ▶ AI가 방향성 없이 추측해서 생성
- ▶ 결과: 일반적인 기본 템플릿
- ▶ 재작업 3~5회 필요

✔ 좋은 프롬프트

프롬프트

오늘 날씨와 주요 뉴스를 보여 주는
나만의 시작 홈페이지를 만들고 싶어.
구글 스타일 디자인으로, 할 일 목록도 포함해 줘.

개선된 점:

- ▶ 목적·스타일·기능이 명확
- ▶ AI가 정확한 방향으로 실행
- ▶ 결과: 맞춤형 홈페이지 완성
- ▶ 1~2회 수정으로 완성

좋은 프롬프트의 6가지 핵심 원칙

Principles of Effective Prompting

1 명확성 (Clarity)

모호함 없이
정확하게 의도 전달

2 구체성 (Specificity)

필요한 세부 사항을
충분히 제공

3 맥락 제공 (Context)

배경 정보로
AI 상황 이해 지원

4 구조화 (Structure)

복잡한 요청은
논리적으로 구성

5 목적 정의 (Purpose)

무엇을, 왜
필요한지 명확히

6 제약 설정 (Constraints)

형식, 길이,
스타일 등 지정

CLEAR 프레임워크

좋은 프롬프트를 만드는 5가지 구성 요소

C

Context (맥락)

VSCode + C++17 CMake 환경 (Java: Maven/Spring Boot 병행)

L

Length (길이)

예제 1개만 / 전체 코드 / 단계별 5개로

E

Examples (예시)

Before/After 코드, 원하는 출력 형태 샘플

A

Ask (요청)

만들어줘 / 분석해줘 / 수정해줘 (동사로 시작)

R

Role (역할)

시니어 C++17 개발자로서 / Java 17 전문가 입장에서



CLEAR 적용 예: "시니어 Java 17 개발자로서(R), VSCode Maven 환경에서(C), 할 일 REST API를 만들어줘(A). GET/POST/DELETE 포함, JUnit 5 테스트까지(E), 전체 코드 완성본으로(L)."

역할 프롬프팅 (Role Prompting)

AI에게 전문가 페르소나를 부여해 전문적 결과 유도

Java 백엔드 개발자

Spring Boot · JPA · RESTful API · 마이크로서비스

C++ 시스템 개발자

STL · RAII · 스마트포인터 · 성능 최적화

코드 리뷰어

버그 탐지 · 컨벤션 검토 · 보안 취약점 분석

소프트웨어 아키텍트

설계 패턴 · 계층 분리 · 확장성 · 의존성 관리

QA 엔지니어

JUnit 5 · Google Test · 경계값 · 통합 테스트

기술 문서 작성자

API 문서 · README · 아키텍처 다이어그램 · JavaDoc

💡 실습: "시니어 Java 17 백엔드 개발자로서, 이 TodoService 코드를 리뷰해 줘. NullPointerException 위험·트랜잭션 처리·성능 이슈 중심으로 검토해 줘."

Prompt Engineering : P-C-T-F & 7단계 프레임워크

AI 응답 품질의 80%는 프롬프트 설계에 달렸다 — 같은 AI라도 프롬프트에 따라 전혀 다른 결과

P-C-T-F 패턴 (Anthropic/IBM 기본 패턴)

- P Persona**
역할 부여
"너는 시니어 백엔드 개발자야"
- C Context**
배경 상황
"현재 주문 시스템 설계 중"
- T Task**
수행 과제
"RESTful API 설계안 3가지 제안"
- F Format**
출력 형식
"각각 장단점을 표로 정리"

7단계 프레임워크 (통합 버전)

- 1 Role & Goal** "기획 전문가, 목표: 요구사항 분석"
- 2 Context & Input** "현재 신규 쇼핑몰 프로젝트"
- 3 Constraints** "300자 이내, 안전 규칙 준수"
- 4 Output Format** "마크다운 표 형식으로"
- 5 Example** "예시: 문제→해결 3가지"
- 6 CoT** "Step by step으로 생각해"
- 7 Feedback Loop** "응답 검토 후 재질문"

→ CRISP 기반 통합: Context+Role → Input+Steps(CoT) → Product

프롬프트 기법 비교 정리

상황별 최적 기법 선택

기법	핵심 원리	사용 상황	효과
P-C-T-F	역할·맥락·작업·형식 4단계	일반 코딩, 분석 요청	★★★★☆
7-Step	목표~반복 7단계 통합	복잡한 기획, 설계 작업	★★★★★
KERNEL	간단·설명·정제·좁힘·탐색·학습	빠른 반복 개선	★★★★☆
Few-Shot	예시를 통한 패턴 학습	특정 형식 반복 출력	★★★★★
CoT	단계별 사고 유도	수학, 논리, 복잡 추론	★★★★★
ReAct	추론·행동·관찰 루프	에이전트 작업, 툴 사용	★★★★☆

KERNE L	영어	핵심 원칙
K	Keep simple	프롬프트를 단순하게 유지하라
E	Explain needs	필요한 것을 명확히 설명하라
R	Refine	결과를 보고 반복적으로 개선하라
N	Narrow	범위를 좁혀서 초점을 맞춰라
E	Explore	다양한 옵션을 탐색하라
L	Learn	예시를 통해 AI가 패턴을 학습하게 하라

할루시네이션 없는 AI 콘텐츠 만들기

AI가 틀린 정보를 자신있게 말하는 현상 — 방지 전략

🚨 할루시네이션 발생 상황

- ▶ 사실 확인 필요한 역사·과학 정보
- ▶ '가장' '최초' '최대' 같은 최상급 표현
- ▶ 특정 시점 기준 수치·통계
- ▶ 코드 API가 실제 존재하는지
- ▶ 외부 라이브러리 버전·메서드명
- ▶ 퀴즈 문제의 정답 정확성

✅ 방지 전략

- ▶ 정답이 하나뿐인지 확인 요청
- ▶ 기준 명시 (2024년 기준, 면적 기준)
- ▶ 2개 이상 출처 교차 검증 요청
- ▶ 논란 내용은 주류 학설 기준 명시
- ▶ 코드는 반드시 실행해서 검증
- ▶ GEMINI.md에 검증 룰 문서화

GEMINI.md 검증 룰

GEMINI.md - 퀴즈 문제 검증 가이드라인

1. 정답이 하나뿐인가? (다른 해석 가능 시 조건 명시)
2. 최상급 표현에 기준이 있는가? ('가장 큰' → 면적 기준)
3. 시간과 범위가 명확한가? (변할 수 있는 정보 시점 명시)
4. 교차 검증했는가? (2개 이상 출처 확인)

고급 프롬프트 전략 — ICL / CoT / ReAct



In-Context Learning (ICL)

Few-shot Prompting

- 학습 없이 예시만 보여줘 동작 유도
- Zero-shot: 예시 없이 지시만
- One-shot: 예시 1개
- Few-shot: 예시 3~5개 → 정확도 ↑

Few-shot 예시

입력: 고객이 화남 → 분류: 부정

입력: 배송이 빠름 → 분류: 긍정

입력: 상품이 마음에 들 → 분류: ?



Chain-of-Thought (CoT)

단계별 사고 유도

- "단계별로 생각해줘" 추가만으로 효과
- 복잡한 수학·논리 문제에 강력
- Zero-shot CoT: "Let's think step by step"
- Few-shot CoT: 풀이 과정 포함 예시 제공

CoT 적용 예

Q: 3명이 일을 나눠서 6일이면

완료. 2명이면 며칠?

→ "단계별로 계산해줘"

Step1: 총 작업량 = $3 \times 6 = 18$ 인일

Step2: 2명이면 $18 \div 2 = 9$ 일



ReAct

Reason + Act

- Reasoning(추론)과 Acting(도구 호출) 결합
- LLM Agent의 핵심 패턴
- Thought → Action → Observation 반복
- Cursor AI / LangChain Agent에서 내부 사용

Thought: 파일 목록을 먼저 확인해야 함

Action: `list_files("./src")`

Observation: `["main.py", "utils.py"]`

Thought: main.py를 읽어야 함

Action: `read_file("main.py")`



Cursor AI는 내부적으로 ReAct 패턴으로 파일 탐색·수정을 수행합니다. 프롬프트에 "단계별로"를 추가하면 CoT가 자동 활성화됩니다.

제로샷 & 퓨샷 프롬프팅

제로샷 (Zero-Shot)

예시 없이 직접 작업 지시
→ 모델이 학습한 지식으로 추론

"이 문장을 영어로 번역해줘:
'오늘 날씨가 정말 좋네요!'"

- ✅ 적합: 명확한 단일 작업
- ❌ 부적합: 복잡한 형식·특정 스타일 요구

퓨샷 (Few-Shot)

2~5개 예시를 함께 제공
→ 패턴 학습 후 동일 형식으로 출력

예시 1: "두 수를 더하는 함수"
→ `def add(a,b): return a+b`

예시 2: "두 수를 곱하는 함수"
→ `def mul(a,b): return a*b`

이제 "두 수의 최댓값 함수"를 작성해줘

- ✅ 적합: 형식·스타일 일관성 필요
- ✅ 코드·문서 생성, 데이터 변환에 강력

생각의 사슬 (CoT) & 2.3.4 페르소나 기법

Chain-of-Thought (CoT)

"step by step으로 생각해"

→ AI가 추론 과정을 단계적으로 서술

"15% 할인 후 20% 추가 할인 총 할인율을 단계별로 계산해"

1단계: $10,000 \times 0.85 = 8,500$ 원

2단계: $8,500 \times 0.80 = 6,800$ 원

∴ **총 32% 할인**

✓ 수학·논리·복잡한 의사결정에 효과적

✓ "Let's think step by step" 문구 추가

페르소나 (Persona) 기법

AI에게 특정 역할·전문성을 부여

→ 일관된 관점·어조·전문 지식으로 응답

 시니어 개발자

"Clean Code 관점에서 리뷰해줘"

 경영 컨설턴트

"McKinsey 방식으로 분석해줘"

 QA 엔지니어

"엣지 케이스를 찾아줘"

 초등학교 선생님

"쉽게 설명해줘"

 ReAct = Reasoning + Acting

Thought → Action → Observation 루프로 자체 교정

사고 제어 기법 (Cognitive Control)

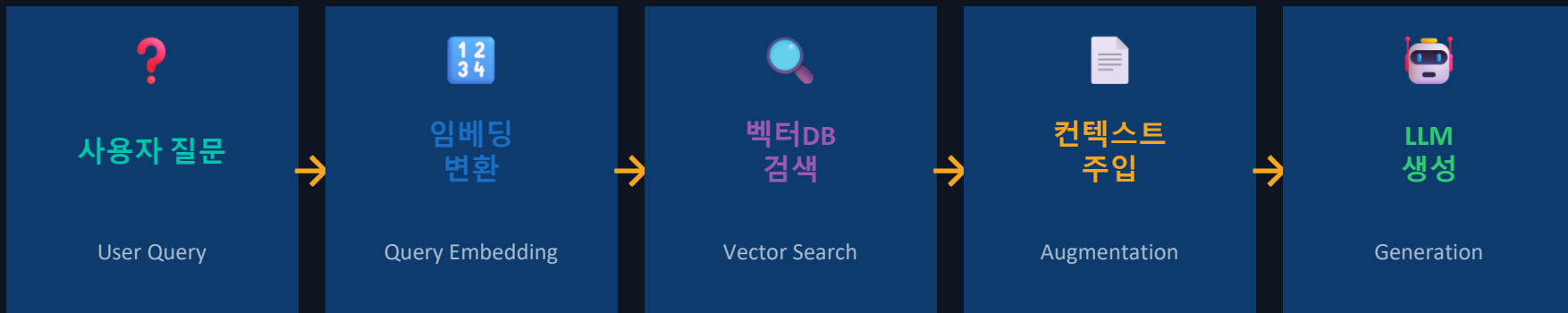
복잡한 문제 해결을 위한 고급 기법

기법	역할	프롬프트 예시
Chain-of-Thought	단계적 추론 유도	'단계별로 생각해서 답해줘'
Few-Shot	예시로 행동 패턴 유도	예시 2-3개 제공 후 질문
Extended Thinking	복잡한 문제에 충분한 시간 부여	'천천히 깊이 생각해줘'
Slow Thinking	가정 검토, 성급한 결론 방지	'결론 내리기 전에 가정을 검토해'
Self-consistency	여러 추론 결과 비교	'3가지 방법으로 풀고 비교해줘'
ReAct	행동-관찰-수정 루프	에이전트 기반 작업 처리

검색 증강 생성 (RAG) — 개요 및 기본 개념

RAG (Retrieval-Augmented Generation)

LLM의 지식 컷오프·환각 문제를 외부 검색으로 보완하는 기술 — 실시간·도메인 특화 정보를 제공



✗ 기존 LLM

학습 데이터 기준 고정 | 최신 정보 없음 | 환각 위험 높음

✓ RAG

실시간 정보 검색 | 출처 제공 | 도메인 특화 가능

RAG 핵심 구성요소 & 실제 적용 사례

 01

Document Loader

PDF·웹페이지·DB에서 텍스트 추출

 02

Text Splitter

긴 문서를 청크(Chunk)로 분할

 03

Embedding Model

텍스트 → 고차원 벡터 변환

 04

Vector Store

벡터 저장 및 유사도 검색
(FAISS·Pinecone)

 05

Retriever

질문과 유사한 문서 청크 추출

 06

LLM + Prompt

검색 결과를 컨텍스트로 최종 답변 생성

실제 적용 사례: Google NotebookLM

- PDF·웹페이지·유튜브를 업로드하면 AI가 내용 기반으로 Q&A 응답 (순수 RAG 구조)
- 환각 최소화: 내 문서 내용만 참조 | 출처 인용 포함 | 오디오 팟캐스트 생성 지원

파인튜닝 (Fine-Tuning)

사전 학습된 대형 모델을 특정 도메인·태스크 데이터로 추가 학습 → 전문화된 성능

전체 파인튜닝

- 모든 파라미터를 재학습
- 최고 성능 달성 가능
- 대규모 GPU·데이터 필요
- 시간·비용 높음
- 적합: GPT-4급 기반 의료·법률 특화 모델

부분 파인튜닝 (LoRA/PEFT)

- 소수 파라미터만 학습 (저랭크 행렬)
- GPU 메모리 90%+ 절감
- 학습 속도 빠름, 성능도 준수
- 원본 모델 보존 가능
- 적합: 스타트업·기업 빠른 프로토타입

파인튜닝 단계 & 활용 사례

파인튜닝 4단계

1

데이터 준비

도메인 특화 데이터 수집·전처리
입력·출력 쌍 구성 (instruction
tuning 형식)

2

기본 모델 선택

GPT-3.5/LLaMA/Gemma 등 오픈소
스 선택
태스크 복잡도·비용 고려

3

학습 실행

하이퍼파라미터 조정 (lr, epoch,
batch)
LoRA 어댑터 또는 전체 파라미터
학습

4

평가 & 배포

벤치마크 평가 → RLHF 추가 개선
API 또는 로컬 배포

활용 사례

의료 챗봇

병원 내부 진료 기록·프로토콜
학습 → 진단 보조

고객 서비스

기업 제품 문서로 학습
→ 자동 CS 응대

법률 분석

판례·계약서 학습
→ 리스크 요약

코드 생성

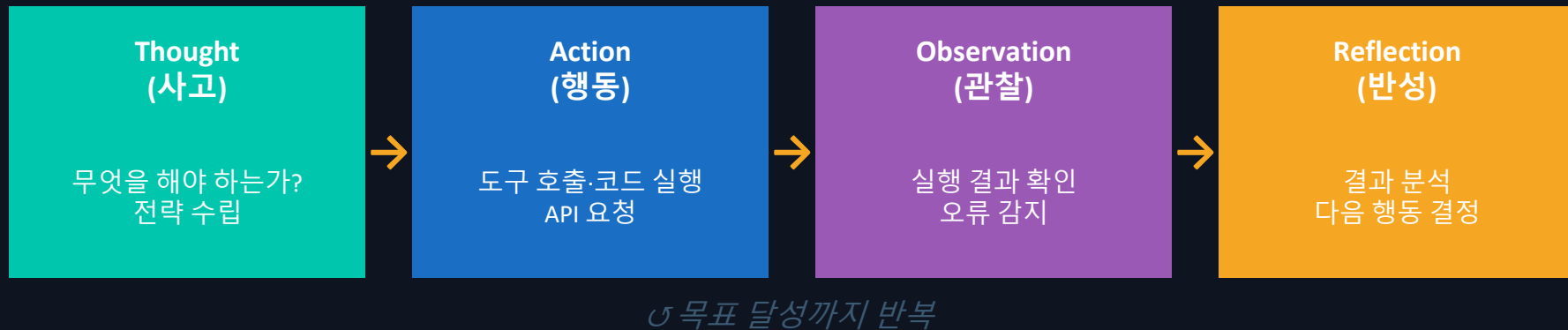
사내 코딩 컨벤션 학습
→ 맞춤형 코드 추천

AI 에이전트 — 정의·원리·사용 사례

AI 에이전트란?

목표를 받으면 스스로 계획·도구 사용·실행·검증을 반복하며 달성하는 자율적 AI 시스템

에이전트 동작 원리 (ReAct Loop)



◆ 코딩 에이전트: Cursor/Cline: 자동 코드 생성→테스트→수정

◆ 업무 자동화: 이메일 분류→CRM 업데이트→일정 조율

◆ 리서치 에이전트: 웹 검색→요약→보고서 자동 작성

◆ MCP 에이전트: DB·API·파일 접근으로 실제 시스템 조작

정리 — RAG vs 파인튜닝 vs 에이전트 비교

구분	RAG	파인튜닝	AI 에이전트
목적	최신 정보 주입	도메인 전문화	자율 태스크 실행
학습 필요	X 불필요	✅ 필요	X 불필요
실시간성	✅ 가능	X 학습 후 고정	✅ 가능
비용	중간	높음	낮음~중간
환각 위험	낮음 (출처 기반)	중간	중간 (도구 오용)
대표 사례	NotebookLM	GPT-4 파인튜닝	Cursor·AutoGPT

🎯 선택 가이드: 최신 정보가 필요하면 → RAG | 특정 스타일·전문성 필요 → 파인튜닝 | 복잡한 자율 작업 → AI 에이전트 | 시스템 연동 → MCP

C++ & Java 개발자 특화 프롬프트 패턴

VSCode 환경에서 언어별로 AI를 200% 활용하는 전략



C++17

효과적인 프롬프트 패턴

- ▶ C++17 modern 스타일로 작성해줘
- ▶ RAII + 스마트 포인터 사용 필수
- ▶ 메모리 누수 없이 구현해 줘
- ▶ STL 알고리즘 최대한 활용해줘
- ▶ Google Test 단위 테스트 포함
- ▶ const correctness 지켜줘



Java 17+

효과적인 프롬프트 패턴

- ▶ Java 17 record/sealed class 활용해줘
- ▶ Spring Boot 3 + Maven 구조로 작성
- ▶ Lombok 없이 순수 Java 17로 작성
- ▶ JUnit 5 + Mockito 테스트도 포함
- ▶ Thread-safe하게 구현해 줘
- ▶ Optional로 null 처리해 줘

✨ 공통 팁: '주석은 한국어로, 변수명은 영어로, 모든 공개 메서드에 예러 처리를 포함해 줘'를 기본 지시로 사용

실습 #1 — 실무 프롬프트 고치기 & 개선하기

과제 1

나쁜 프롬프트 개선하기

🕒 10분

❌ 나쁜 프롬프트

"로그인 기능 만들어줘"

✅ 개선 방향

P-C-T-F로 개선:

[P] 너는 시니어 백엔드 개발자야

[C] 이커머스 플랫폼, ID+이메일 로그인 지원

[T] JWT 기반 로그인 API 설계 및 코드 작성

[F] Swagger 형식 + Python FastAPI 코드

과제 2

7-Step Framework 작성

🕒 20분

❌ 나쁜 프롬프트

이커머스 장바구니 시스템 설계

✅ 개선 방향

7단계 적용:

① Role: 소프트웨어 설계자

② Context: 기존 시스템의 4가지 문제점

③ Task: 7단계 구조로 분석·설계

④ Constraints: UI·DB 제외, 비즈니스 규칙 명확

⑤ Output: 구조화된 Markdown 형식

⑥ CoT: step by step 사고

⑦ Example: 문제→해결 3가지

실습 #1 — 실무 프롬프트 고치기 & 개선하기

과제 3 Few-Shot 예시 작성

🕒 20분

❌ 나쁜 프롬프트

데이터 유효성 검증 함수 만들어줘

✅ 개선 방향

Few-Shot 구조:

예시1: 이메일 유효성 검증 함수

예시2: 전화번호 유효성 검증 함수

예시3: 나이 범위 검증 함수

→ 형식: 입력조건·정상케이스·실패케이스

반환형식 일관성 유지

실제요청: 주민번호 앞자리 검증 함수

실습 #2 — AI로 나만의 첫 홈페이지 만들기

Gemini CLI로 C++ & Java 3단계 반복 개선 실습

Step 1

"오늘 날씨와 주요 뉴스를 보여 주는 나만의 시작 홈페이지를 만들고 싶어"

Step 2

"구글 스타일로 디자인을 수정해 줘."

Step 3

"오늘의 할 일 목록을 추가해 줘."

GPT 병행 실습

- ▶ "HTML로 나를 소개하는 홈페이지를 간단하게 만들어줘."
- ▶ "버튼을 클릭하면 로또 번호를 추천하는 웹 사이트 코드를 만들어 주세요."

학습 포인트

- ▶ 반복 프롬프트로 점진적 품질 향상
- ▶ Gemini vs Cursor AI 결과 비교 분석
- ▶ 무료 메시지 한도 관리 전략

💡 팁: Gemini CLI 무제한 무료 사용 — C++ CMake 프로젝트 생성부터 Java Spring Boot까지 터미널에서 바로 실습

실습 #3 — PRD(제품 요구사항 문서) 작성하기

코딩 전 AI가 정확하게 실행할 수 있도록 명세서를 먼저 작성

PRD가 필요한 이유

- ▶ 방향성 없는 코딩 방지
- ▶ 명확한 기능 범위 설정
- ▶ 재작업 횟수 70% 감소
- ▶ 단계별 구현 계획 수립
- ▶ 팀 협업 공통 기반 제공

PRD 구성 요소

- ① 프로젝트 개요 (목적·배경)
- ② 타겟 사용자 정의
- ③ 핵심 기능 (Must/Should)
- ④ 기술 스택·제약 조건
- ⑤ 성공 지표 (KPI)

프롬프트

포트폴리오 웹페이지를 만들려고 해.
주요 기능은 채용 담당자가 30초 안에 내
역량을 파악할 수 있는 임팩트 있는 구성
이야.
프리랜서 전환을 준비하는 마케터를 위
한 것이고, 경쟁력 있는 지원자로 보이기
위해 필요해.
기술적인 방향과 함께 구체적인 PRD를
작성해 줘.

📌 실습 순서: PRD 작성 → 단계별 분해 → 단계별 실행 → 검토 요청 → 개선 요청 (5단계 패턴)

단계별 프롬프트 전략

큰 작업을 5단계로 쪼개면 AI가 더 정확하게 실행한다

1

구조 설계

파일 구조·아키텍처만
먼저

2

핵심 기능

비즈니스 로직 구현

3

UI/UX

디자인·스타일·인터랙
션

4

고급 기능

검색·다크모드·단축키

5

최적화

성능·접근성·에러 처리

💡 단계 분해 프롬프트: "이 PRD를 Gemini CLI / Cursor AI에서 사용할 단계별 프롬프트로 변환해 줘. 5개의 핵심 단계로 정리해서 각 단계마다 명확한 지시 사항을 만들어 줘."

단계 분해 결 과 예시

```
# 포트폴리오 PRD 단계별 분해 결과
Step1: HTML 전체 구조 + 섹션 이름 부여 (기능 구현 전)
Step2: performance-metrics 섹션 내용 구체화
Step3: 모던한 CSS 디자인 적용
Step4: 차트·인터랙션 기능 추가
Step5: PRD 기준 검토 + 발견된 개선점 3가지 수정
```

반복.개선 프롬프트 패턴

한 번에 완벽하지 않아도 OK — 대화하며 점진적 품질 향상

이터레이션

점진적 개선

"구글 스타일로 디자인을 수정해 줘."

검토 요청

품질 검증

"PRD대로 제작되었는지, 링크가 작동하는지 확인해 줘."

파일 참조

컨텍스트 전달

"@script.js 파일에서 다크모드 함수를 찾고 설명해 줘."

섹션 지정

특정 부분만

"case-studies 섹션을 hero-introduction으로 옮겨 줘."

보완 요청

선택적 수정

"발견된 개선점 3가지를 보완하고 최종 완성해."

협업 검토

폴더 참조

"@web_version 폴더를 분석해서 데스크톱 최적화 확인해 줘."

✓ 핵심: AI와의 대화는 한 번에 끝내려 하지 말 것 — 대화할수록 결과가 좋아진다

1시간 정리

프롬프트 엔지니어링 핵심 요약

지금까지 배운 6가지 핵심 원칙 확인



CLEAR 프레임워크

Context · Length · Examples · Ask · Role 5요소



PRD 먼저 작성

코딩 전 요구사항 문서화 → 재작업 70% 감소



단계별 분해

큰 작업 → 5단계로 쪼개 순차 실행



반복 개선 패턴

이터레이션·섹션지정·검토·보완 4패턴



역할 부여

전문가 페르소나로 전문적 답변 유도



할루시네이션 방지

GEMINI.md 검증 룰 + 교차 확인 요청



다음 시간: Gemini CLI 설치 & 기초 실습 — 터미널 AI 개발 환경 구축 + 손글씨 인식 앱 완성

2번째 시간

Gemini CLI 설치 & 기초 실습

터미널 기반 AI 코딩 환경 구축 → 손글씨 인식 앱 완성



프롬프트 기반 코딩의 진화

코딩 패러다임의 혁명적 변화

"AI가 코더를 대체하지는 않지만, AI를 활용하는 코더가 그렇지 않은 코더를 대체할 것입니다."

~2023

전통적 코딩

모든 코드를
수작업으로 작성

개발자가 직접
문법·API 암기

2024

AI 보조 코딩

GitHub Copilot,
Tabnine 등

AI가 부분적으로
코드 보완

2025~

AI 에이전트 코딩

Cursor, Cline

AI가 모든 코드를
완전 자동 생성

개발자 = 설계자

기존 개발 vs 바이트 코딩

기존 개발 방식

직접 코드 작성



한 줄씩 타이핑 → 문서 검색 → 스택오버플로우 반복

수동 디버깅



에러 메시지 → 검색 → 원인 추론 → 수정 → 반복

파일별 편집



에디터에서 파일을 열고 닫으며 컨텍스트 전환

긴 피드백 루프



코드 작성 → 빌드 → 테스트 → 수정 cycle이 느림

지식 검색



API 문서, 예제 코드를 직접 찾아보는 시간 낭비

바이트 코딩 방식

의도 중심 지시



"로그인 기능 만들어줘" → AI가 파일 생성·수정 자동화

AI 자율 디버깅



에러 발생 시 AI가 원인 분석 → 수정 → 검증 루프 자동

프로젝트 전체 이해



AI가 전체 코드베이스 파악 후 관련 파일 동시 수정

즉각적 피드백



지시 → 코드 생성 → 실행 → 결과 확인까지 수분 내 완료

검증 중심 역할



개발자는 '무엇을 만들지'에 집중, AI는 '어떻게'를 담당

주요 AI 코딩 에이전트 비교

2025년 기준

도구	특징	강점	가격
Cursor AI	VS Code 기반 IDE, 멀티파일 편집	Agent/Plan/Ask 3모드, .cursorsrules	Free / \$20~
GitHub Copilot	GitHub 공식 통합, 모든 IDE 지원	기업 보안 강화, 대형 레포 최적화	\$10~\$19/월
Cline	오픈소스, VS Code 확장	SNS 인기, 직접 API 사용 가능	Free (API 비용)
Amazon Q	AWS 생태계 최적화	AWS 서비스 연동, 기업용	\$19/월
Claude (Anthropic)	Artifacts, Projects 기능	긴 컨텍스트, 문서 작업 특화	Free / \$20~
Gemini CLI	Google	1M 토큰 컨텍스트, Google Drive 연동 , 무료 티어 넉넉	
OpenAI Codex CLI	OpenAI	GPT-4o 기반, 함수 단위 완성 강점, GitHub Copilot 연계	

Cursor AI 요금제 비교 (2025 기준)

가입 즉시 사용 가능, Pro 14일 무료 체험

요금제	가격	자동 완성	프리미엄 요청	비고
Free	무료	월 200~2000회	월 50회 (느림)	2주간 Pro 체험 포함
Pro	\$20/월	무제한	월 500회 (빠름) + 느림 무제한	GPT-4, Claude 사용 가능
Business	\$40/월/인	Pro 기능 전체	Pro + 조직용 보안·대시보드	팀 사용 추천

AI 코딩 도구 비교 & 선택 가이드

각 도구의 강점을 파악하고 상황에 맞게 선택



Gemini / ChatGPT

gemini.google.com /
chat.openai.com

✓ PRD 작성·코드 설명·긴 컨텍스트 처리 (보조 도구)

⚠ 무료 한도 있음



Gemini CLI

`npm i -g @google/gemini-cli`

✓ 터미널 기반 개발·대용량 파일·무료

⚠ 초기 설정 필요



Cursor AI

cursor.sh (월 \$20)

✓ IDE 통합·전체 코드베이스·Tab 자동완성

⚠ 유료 플랜 필요



Firebase Studio

firebase.studio

✓ Gemini 통합 웹 기반 개발 환경

⚠ Google 계정 필요



이 과정: Gemini CLI (C++/Java 터미널 개발) + Cursor AI (IDE 개발) — C++ 중심, Java 병행 실습

Gemini CLI란? — 터미널 AI 개발 도구

구글의 무료 터미널 기반 AI 코딩 어시스턴트

◆ Gemini CLI 특징

- ▶ VSCode 통합 터미널에서 실행
- ▶ 1M 토큰 대용량 컨텍스트 지원
- ▶ Google 계정으로 무료 사용
- ▶ 파일 시스템 직접 접근·수정
- ▶ 이미지·코드 멀티모달 처리
- ▶ Firebase Studio 연동 지원

AI CLI 도구 비교

Gemini CLI

```
bash
```

```
npm i -g @google/gemini-cli
```

Gemini CLI

```
bash
```

```
npm i -g @google/gemini-cli
```

OpenAI Codex

```
bash
```

```
npm i -g @openai/codex
```

📌 이 과정의 핵심 도구: Gemini CLI — 무료이면서 강력한 터미널 AI 개발 환경
<https://geminicli.com/> | <https://aistudio.google.com/welcome>

Node.js & Gemini CLI 설치

nodejs.org → npm install → gemini 명령어 확인

bash

1. Node.js 설치 확인 (nodejs.org/ko 에서 LTS 버전 설치)

```
C:\Users\dev> node --version
```

```
v22.22.2
```

```
C:\Users\dev> npm --version
```

```
10.9.7
```

2. Gemini CLI 전역 설치

```
npm install -g @google/gemini-cli
```

3. 설치 확인

```
gemini --version
```

4. 첫 실행 (브라우저에서 Google 계정 로그인)

```
gemini
```

5. VSCode 통합 터미널에서 사용 (Ctrl + 백틱)

설치 성공 확인

- ▶ gemini 명령어 실행 → 대화 프롬프트 나타남
- ▶ 구글 계정 로그인 완료

트러블슈팅

- ▶ npm 오류 → Node.js LTS 재설치
- ▶ 인증 실패 → gemini auth login 재실행

실습 — Build > debug & deploy with AI

터미널에서 AI와 효과적으로 대화하기 위한 환경설정

Node.js

<https://nodejs.org/ko>

GeminiCLI

<https://geminicli.com/>

```
npm install -g @google/gemini-cli
```

Codex

<https://openai.com/ko-KR/codex/>

```
npm i -g @openai/codex
```

Claude Code

<https://code.claude.com/docs/ko/setup>

실습 — 손글씨 숫자 인식 프로그램

Gemini CLI로 Python + 실행 파일 + 웹/데스크톱 버전 완성

Step 1

"손글씨로 숫자를 입력하면 이것을 인식하는 코드를 만들어서 실행해 줘. 모든 코드와 주석을 영어로 작성해 줘."

Step 2

"숫자 인식 프로그램을 윈도우 탐색기에서 클릭해서 실행할 수 있게 해 줘."

Step 3

"손글씨 인식 프로그램을 웹 버전과 데스크톱 버전으로 나누어 개발하고 싶어. web_version과 desktop_version 폴더를 만들어 줘."

tree

Gemini CLI가 자동 생성하는 구조

HandwritingRecog/

```
├── GEMINI.md           # 프로젝트 컨텍스트
├── recognizer.py       # 메인 손글씨 인식
├── train.py            # MNIST 모델 학습
├── model/              # 저장된 모델 파일
├── web_version/
│   ├── GEMINI.md      # 웹 버전 전용 설정
│   └── index.html      # 브라우저 실행
└── desktop_version/
```

💡 PyInstaller로 .exe 생성: Gemini CLI에게 "실행 파일로 패키징해줘"라고 하면 자동으로 pyinstaller 설치 및 빌드

GEMINI.md — 프로젝트 컨텍스트 파일

AI가 프로젝트를 이해할 수 있도록 핵심 정보를 문서화

markdown

```
# GEMINI.md - 프로젝트 루트에 생성
# Project: Handwriting Number Recognition

## Environment
- Language: Python 3.11 + Java 17 / C++17
- IDE: VSCode with Gemini CLI
- Build: Maven (Java) / CMake (C++)

## Code Rules
- 모든 파일 상단에 날짜와 시간 주석 필수
- 변수명: camelCase (Java), snake_case (C++)
- 에러 처리: 모든 공개 메서드에 예외 처리
- 테스트: JUnit 5 / Google Test 단위 테스트 포함

## Architecture
- web_version/      → 브라우저 실행 버전
- desktop_version/ → 데스크톱 앱 버전

## Current Task
손글씨 숫자 인식 모델 개선 중 (목표 정확도 95%)
```

프롬프트

```
# 앞으로 만드는 모든 파일은 날짜와 시간을 주석으로 표시해 줘.
손글씨 인식 프로그램을 웹 버전과 데스크톱 버전으로 나누어 개발하고 싶어.
web_version과 desktop_version 폴더를 만들고, 각 폴더에 해당하는 GEMINI.md 파일도 생성해 줘.
```

Gemini CLI 주요 명령어 설명

`/clear`: 현재 화면에 가득 찬 대화 내용을 깨끗이 지우고 새로운 세션(대화)을 시작할 때 사용합니다. 작업 내용이 너무 복잡해졌을 때 유용합니다.

`/quit`: 현재 실행 중인 Gemini CLI 프로그램을 종료하고 일반 터미널 상태로 돌아갑니다.

`/directory`: 현재 AI가 작업하고 있는 폴더(작업 디렉토리)를 관리합니다. 다른 프로젝트 폴더로 이동하거나 현재 위치를 확인할 때 씁니다.

`/compress`: 대화 내용이 너무 길어지면 AI가 이전 내용을 잊어버릴 수 있습니다. 이때 이 명령어를 쓰면 지금까지의 내용을 요약해서 기억 공간을 효율적으로 관리해 줍니다.

`/tasks`: 현재 백그라운드에서 실행 중인 작업(예: 패키지 설치, 서버 구동 등)이 있는지 목록을 켜다(Toggle) 하며 확인할 수 있습니다.

`/stats`: 현재 사용 중인 AI 모델 정보, 도구 사용 횟수, 세션 시간 등 통계 정보를 보여줍니다.

`/footer`: 화면 하단 상태 표시줄(statusline)에 어떤 항목을 보여줄지 설정합니다.

`/about`: 현재 설치된 Gemini CLI의 버전 정보를 확인합니다.

Gemini CLI 실전 팁 & 트러블슈팅

효율을 높이는 고급 사용법과 자주 발생하는 문제 해결

⚡ 생산성 향상 팁

- ▶ GEMINI.md로 매번 컨텍스트 반복 방지
- ▶ 긴 프롬프트는 .md 파일로 저장 후 호출
- ▶ 작업 단위를 작게 나눠 순차 실행
- ▶ @파일명으로 특정 파일만 참조
- ▶ ! 명령으로 셸 결과 즉시 피드백
- ▶ /clear 후 재시작 (대화 너무 길면)

🔧 트러블슈팅 Q&A

- | | |
|-----------|--------------------------|
| ⚠ 인증 실패 | → gemini auth login 재실행 |
| ⚠ 응답 느림 | → gemini-2.0-flash 모델 사용 |
| ⚠ 컨텍스트 초과 | → 프로젝트 작은 단위로 분할 |
| ⚠ 한글 깨짐 | → 터미널 인코딩 UTF-8 설정 |
| ⚠ npm 오류 | → Node.js LTS 버전 재설치 |

✅ 핵심: AI가 실수해도 OK — 반복적으로 수정 요청하면 결국 원하는 결과가 나온다

VSCode 통합 터미널 최적화 설정

Gemini CLI를 VSCode에서 가장 효율적으로 사용하는 환경

json

```
// .vscode/settings.json - 추천 설정
{
  "terminal.integrated.defaultProfile.windows": "Command Prompt",
  "terminal.integrated.fontSize": 14,
  "editor.formatOnSave": true,
  "editor.tabSize": 4,

  // C++ 설정
  "C_Cpp.default.cppStandard": "c++17",
  "cmake.buildDirectory": "${workspaceFolder}/build",
  "clangd.arguments": ["--header-insertion=never"],

  // Java 설정
  "java.compile.nullAnalysis.mode": "auto",
  "java.test.defaultConfig": "junit5"
}

// 추천 VSCode 확장 설치
// C++: C/C++ Extension Pack, CMake Tools, clangd
// Java: Extension Pack for Java, Maven, Gradle
```

Ctrl + 백틱

통합 터미널 열기/닫기

Ctrl+Shift+5

터미널 분할 (Gemini + 빌드)

Ctrl+Shift+P

Command Palette (설정 검색)

3번째 시간

Cursor AI 설치 & 코드 품질

IDE 통합 AI — 코드베이스 전체를 이해하는 AI 에디터



Cursor AI란? — VSCode 기반 AI IDE

전체 코드베이스를 이해하는 AI가 내장된 코드 에디터

Cursor AI 핵심 기능

- ▶ 전체 코드베이스 컨텍스트 이해
- ▶ Tab 자동완성 (다음 코드 AI 예측)
- ▶ Ctrl+K: 코드 인라인 직접 수정
- ▶ Ctrl+L: 채팅으로 코드 생성·설명
- ▶ Ctrl+Shift+L: 에이전트 모드
- ▶ @파일/@폴더로 컨텍스트 참조

Gemini CLI vs Cursor AI

Gemini CLI

무료

터미널 기반, 파일 직접 생성·수정

Cursor AI

월 \$20

IDE 통합, 실시간 코드 편집·자동완성

 설치: cursor.sh 에서 다운로드 → 기존 VSCode 설정·확장·단축키 그대로 이전 가능

Cursor AI — 설치 & 핵심 설정

🌸 설치 & 초기 설정 (cursor.sh)

설치 순서

```
# 1. cursor.sh 에서 다운로드 & 설치
#   (Windows / macOS / Linux 지원)

# 2. 기존 VSCode 설정 Import
#   File > Preferences > Import Settings
#   → extensions, keybindings 자동 이전

# 3. 로그인 (GitHub / Google OAuth)
#   Cursor > Sign In

# 4. AI 모델 선택
#   Settings > Models
#   → claude-3-5-sonnet, gpt-4o 등 선택
```

📁 .cursorrules — 프로젝트 AI 설정 파일

.cursorrules (프로젝트 루트)

```
# 기술 스택
- TypeScript + Node.js 20, Next.js 14
- Prisma ORM + PostgreSQL
- 코드 스타일: ESLint + Prettier (2스페이스)

# AI 지시 규칙
- console.log 금지 → winston logger 사용
```

📄 4가지 핵심 모드 & 단축키

Ctrl+L

Chat

사이드바 대화 — @파일·@폴더·@코드베이스 참조 가능

Tab

자동완성

다음 줄 예측 완성 — Ghost Text 수락/거절

Cmd+K

인라인 편집

선택 코드 위에서 직접 편집 지시 → 즉시 반영

Ctrl+I

Composer

여러 파일에 걸친 대규모 변경 — 새 기능 추가에 최적

Ctrl+Shift+J

Cursor Rules

프로젝트 Rules 설정 파일 열기 & 편집

@기호

컨텍스트 참조

@파일 @폴더 @웹 @Docs — Chat에서 직접 참조

/명령어

슬래시 커맨드

/review /explain /fix /test 등 빠른 실행

Ctrl+Shift+L

전체 코드 선택

현재 파일 전체를 Chat 컨텍스트로 추가



.cursorrules는 팀 전체가 공유하는 AI 설정입니다. Git에 커밋해 팀원 모두가 동일한 AI 행동을 경험하게 하세요.

.cursorrules — 팀 코딩 규칙 자동 적용

프로젝트 루트에 두면 모든 AI 응답에 규칙이 자동 적용

text

```
## C++17 Rules
```

```
You are a senior C++17 developer.
```

```
Always use:
```

- Smart pointers (unique_ptr, shared_ptr) instead of raw pointers
- RAII for resource management
- const correctness throughout
- STL algorithms over manual loops
- Google Test for unit tests
- snake_case for variables, PascalCase for classes

```
## General Rules
```

- Add date/time comment at the top of every new file
- Handle all edge cases with proper error messages
- Write thread-safe code when dealing with shared state

🌟 .cursorrules 한 번 설정 → 이후 모든 코드가 팀 규칙을 자동으로 따름 — 코드 리뷰 지적 70% 감소

.cursorrules — 팀 코딩 규칙 자동 적용

프로젝트 루트에 두면 모든 AI 응답에 규칙이 자동 적용

text

```
# .cursorrules 파일 (프로젝트 루트)

## Java 17 Rules
You are a senior Java 17 developer.
Always use:
- Java 17 Records for DTOs (not traditional classes)
- Spring Boot 3 with Maven (not Gradle unless specified)
- JUnit 5 + Mockito for unit tests
- Optional<T> for null handling (never return null)
- Stream API for collection operations
- Korean comments, English variable/method names
- Proper exception handling with @ControllerAdvice
```

🌟 .cursorrules 한 번 설정 → 이후 모든 코드가 팀 규칙을 자동으로 따름 — 코드 리뷰 지적 70% 감소

Cursor AI — C++ 코드 품질 분석 & 현대화

AI가 C++ 코드베이스 전체를 분석해 스마트 포인터로 자동 리팩토링

cpp

```
// Cursor AI Ctrl+L 채팅 프롬프트:  
// "@TodoService.cpp 이 파일의 메모리 관리 개선점을 알려줘"  
  
// ❌ Before - Cursor가 문제 발견  
class TodoService {  
    vector<Todo*> todos;           // 원시 포인터 → 메모리 누수 위험  
public:  
    void add(Todo* todo) { todos.push_back(todo); }  
    ~TodoService() {  
        for (auto t : todos) delete t; // 예외 시 누수  
    }  
};
```

⚙️ C++ 개발자 특화: Cursor가 RAII·스마트 포인터·const correctness·STL 알고리즘을 자동으로 현대화

Cursor AI — C++ 코드 품질 분석 & 현대화

AI가 C++ 코드베이스 전체를 분석해 스마트 포인터로 자동 리팩토링

cpp

```
// ✅ After – Cursor AI 리팩토링 제안 수락 (Tab)
class TodoService {
    vector<unique_ptr<Todo>> todos; // RAII 자동 메모리 관리
public:
    void add(unique_ptr<Todo> todo) {
        todos.push_back(std::move(todo));
    }
    // 소멸자 불필요 – unique_ptr이 자동 해제
};

// Google Test 자동 생성 (Ctrl+L: "이 클래스 단위 테스트 만들어줘")
TEST(TodoServiceTest, AddAndRetrieve) {
    TodoService service;
    service.add(make_unique<Todo>(1, "Test", false));
    ASSERT_EQ(service.findAll().size(), 1);
}
```


Cursor AI 실습 — Java Spring Boot REST API

@Rules 설정 후 TodoController 자동 생성

java

```
// Cursor AI Ctrl+L 대화로 생성  
// 프롬프트: "Java 17 Spring Boot 3으로 Todo REST API 만들어줘. @Rules 적용"
```

```
@RestController  
@RequestMapping("/api/todos")  
@RequiredArgsConstructor  
public class TodoController {  
    private final TodoService todoService;  
  
    @GetMapping  
    public ResponseEntity<List<TodoResponse>> findAll(  
        @RequestParam(required=false) Category category) {  
        return ResponseEntity.ok(todoService.findAll(category));  
    }  
}
```

💡 Ctrl+L: "@TodoService.java NullPointerException 방지하고 Optional 사용해줘" → 자동 리팩토링

Cursor AI 실습 — Java Spring Boot REST API

@Rules 설정 후 TodoController 자동 생성

java

```
@PostMapping
public ResponseEntity<TodoResponse> create(
    @RequestBody @Valid CreateTodoRequest request) {
    return ResponseEntity.status(201).body(todoService.create(request));
}

@PutMapping("/{id}")
public ResponseEntity<TodoResponse> update(
    @PathVariable Long id,
    @RequestBody @Valid UpdateTodoRequest request) {
    return ResponseEntity.ok(todoService.update(id, request));
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
    todoService.delete(id);
    return ResponseEntity.noContent().build();
}
}
```

Cursor AI — 코드 설명 & 단위 테스트 자동 생성

복잡한 코드 이해 + JUnit 5 / Google Test 자동 생성

java

```
// Ctrl+L 활용 예시 1: 코드 설명 요청
// "@script.js 파일에서 다크 모드 관련 함수를 찾고 설명해 줘."

// Ctrl+L 활용 예시 2: JUnit 5 테스트 자동 생성 (Java)
// "@TodoService.java 의 모든 public 메서드에 대한 JUnit 5 테스트를 만들어줘"

@ExtendWith(MockitoExtension.class)
class TodoServiceTest {
    @Mock private TodoRepository todoRepository;
    @InjectMocks private TodoService todoService;
```

💡 Cursor의 강점: @파일명으로 실제 코드를 컨텍스트에 포함 → 코드를 완전히 이해한 상태로 테스트 생성

Cursor AI — 코드 설명 & 단위 테스트 자동 생성

복잡한 코드 이해 + JUnit 5 / Google Test 자동 생성

java

```
@Test
@DisplayName("할 일 추가 시 저장소에 저장됨")
void create_shouldSaveTodoToRepository() {
    // Given
    var request = new CreateTodoRequest("테스트 할 일", Category.WORK);
    var expected = new Todo(1L, "테스트 할 일", false, Category.WORK, LocalDateTime.now());
    when(todoRepository.save(any())).thenReturn(expected);

    // When
    var result = todoService.create(request);

    // Then
    assertThat(result.text()).isEqualTo("테스트 할 일");
    verify(todoRepository, times(1)).save(any());
}
```

Cursor AI — 데스크톱 UI 최적화 실습

모바일 최적화된 Todo 앱을 데스크톱 환경으로 개선

프롬프트

[현재 디자인은 모바일에 최적화되어 있는 것 같아]
데스크톱 환경에서도 전체 화면에서 UI를 볼 수 있도록 디자인을 수정해 줘.
수정한 디자인은 'web_version'이라는 새로운 폴더를 생성한 뒤, 그곳에 저장해 줘.

CSS

```
/* Cursor AI가 수정한 데스크톱 CSS */  
  
/* Before: 모바일 중심 */  
.container { max-width: 600px; margin: 0 auto; }
```

💡 Cursor에 이미지를 첨부하면 '이 디자인처럼 수정해줘'도 가능 — 시각적 레퍼런스 기반 개발

Cursor AI — 데스크톱 UI 최적화 실습

모바일 최적화된 Todo 앱을 데스크톱 환경으로 개선

CSS

```
/* After: 데스크톱 반응형 2열 레이아웃 */
.app-layout {
  display: grid;
  grid-template-columns: 300px 1fr; /* 사이드바 + 메인 */
  gap: 24px;
  max-width: 1400px;
  margin: 0 auto;
  padding: 24px;
}

@media (max-width: 768px) {
  .app-layout { grid-template-columns: 1fr; } /* 모바일 1열 */
}

/* 대시보드: 4열 통계 카드 */
.stats-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 16px;
}
```

Cursor AI — @폴더 분석 & 전체 코드베이스 이해

프로젝트 전체를 AI가 이해한 상태로 최적화 제안

프롬프트

@web_version 폴더에 있는 파일들을 분석해서 할 일 관리 앱이 데스크톱 환경에 최적화되었는지 확인해 줘.

javascript

```
// Cursor AI 분석 결과 예시:  
// @web_version 폴더 전체 스캔 완료  
//  
// 발견된 이슈:  
// 1. style.css: max-width: 600px → 데스크톱에서 공간 낭비  
// 2. script.js: 키보드 단축키 없음 → 데스크톱 사용성 저하  
// 3. index.html: 사이드바 레이아웃 없음 → 단일 컬럼만 존재  
//
```

🌟 @폴더 참조의 힘: AI가 폴더 내 모든 파일을 분석 → 일관된 수정 제안 → 에이전트로 한 번에 적용

Cursor AI — @폴더 분석 & 전체 코드베이스 이해

프로젝트 전체를 AI가 이해한 상태로 최적화 제안

javascript

```
// 권장 수정 사항:
// ✅ CSS Grid 2열 레이아웃 적용
// ✅ 데스크톱 전용 단축키 추가 (Alt+N, Alt+1~4)
// ✅ 와이드 스크린 대시보드 (4열 통계 카드)
// ✅ 사이드바에 카테고리 필터 배치
//
// > 이 수정 사항을 모두 적용해줘. (Ctrl+Shift+L 에이전트)
//
// 에이전트 실행 결과:
// ✅ web_version/index.html 수정 (사이드바 추가)
// ✅ web_version/style.css 수정 (Grid 레이아웃)
// ✅ web_version/script.js 수정 (단축키 추가)
```


Cursor AI — C++17 현대적 패턴 자동 적용

std::variant · std::optional · ranges · structured bindings

cpp

```
// Cursor AI 생성 — C++17 modern 패턴
// Ctrl+L: "C++17 std::variant로 오류 처리 패턴 만들어줘"

#include <variant>
#include <optional>
#include <string>

// std::variant로 Result 타입 구현
template<typename T>
using Result = std::variant<T, std::string>; // Success | Error

Result<Todo> TodoService::findById(long id) const {
    auto it = std::find_if(todos_.begin(), todos_.end(),
        [id](const Todo& t){ return t.id == id; });
    if (it == todos_.end())
        return std::string("Todo not found: " + std::to_string(id));
    return *it;
}
```

Cursor AI — C++17 현대적 패턴 자동 적용

std::variant · std::optional · ranges · structured bindings

cpp

```
// Structured bindings (C++17)
auto [todos, error] = service.findAll();

// std::optional 활용
std::optional<Todo> TodoService::findFirst() const {
    if (todos_.empty()) return std::nullopt;
    return todos_.front();
}

// Ranges (C++20 preview)
auto workTodos = todos_ | std::views::filter(
    [](const Todo& t){ return t.category == Category::WORK; });
```

🌸 C++17 = Java Optional(std::optional) + Java Result(std::variant) + Java Stream(std::ranges) — Cursor가 자동 대응

Cursor AI 실습 — Java 17 고급 패턴 적용

Record · Sealed Class · Pattern Matching 자동 생성

java

```
// Cursor AI Ctrl+L: "Java 17 sealed class로 Todo 응답 타입 만들어줘"
```

```
// Sealed Class + Pattern Matching (Java 17)
```

```
public sealed interface TodoResult
    permits TodoResult.Success, TodoResult.Error {

    record Success(TodoResponse data) implements TodoResult {}
    record Error(String message, int code) implements TodoResult {}
}
```

```
// Pattern Matching으로 처리
```

```
public ResponseEntity<?> handleResult(TodoResult result) {
    return switch (result) {
        case TodoResult.Success s -> ResponseEntity.ok(s.data());
        case TodoResult.Error e   -> ResponseEntity
            .status(e.code()).body(Map.of("error", e.message()));
    };
}
```

Cursor AI 실습 — Java 17 고급 패턴 적용

Record · Sealed Class · Pattern Matching 자동 생성

java

```
// Cursor Tab 자동완성 - 다음 줄 예측
// '/// 카테고리별 통계 조회' 입력 후 Tab:
public Map<Category, Long> getCategoryStats() {
    return todoRepository.findAll().stream()
        .collect(Collectors.groupingBy(
            Todo::category, Collectors.counting()));
}
```

 Cursor Tab 자동완성: 주석을 작성하면 AI가 구현 코드를 예측 → Tab으로 수락 → 반복 코드 80% 감소

Cursor AI 핵심 워크플로우 6패턴

1

코드 탐색 & 이해

@Codebase 이 프로젝트의 전체 구조를 설명하고
주요 데이터 흐름을 Mermaid 다이어그램으로 그려줘.
→ 파일 트리 분석 → 의존성 맵 → 다이어그램 Chat에 표시

2

기능 구현 (Composer)

Ctrl+I → users 테이블에 avatar_url 추가하고
프로필 이미지 업로드 API를 만들어줘.
→ 마이그레이션+API 라우트+타입 정의 동시 생성

3

버그 수정 (Chat)

@terminal 에러 로그를 보고 원인 분석해줘.
@UserService.ts 42번째 줄을 수정해줘.
→ 에러 분석 → 원인 파악 → Apply 클릭 한 번으로 수정

4

인라인 리팩토링

Cmd+K → (auth 모듈 선택 후)
중복 코드 제거하고 SOLID 원칙 적용해줘.
→ 선택 범위 내 즉각 변경 → Diff로 확인 후 수락

5

문서화 자동 생성

@src/routes 폴더의 모든 API에
JSDoc + OpenAPI 3.0 주석을 추가해줘.
→ Composer가 라우트 파일 전체 순회 → 주석 일괄 추가

6

보안 감사

@Codebase OWASP Top 10 기준으로
취약점 스캔하고 수정 우선순위 알려줘.
→ Chat에 취약점 목록 + 심각도 + 수정 코드 스니펫 제시

Cursor AI 기초 & 코드 품질 정리

IDE 통합 AI로 개발 속도 3배 향상 확인

● Cursor AI 설치

cursor.sh 다운로드 → VSCode 설정 이전

● .cursorrules

C++/Java 팀 규칙을 AI에게 영구 지시

● Tab 자동완성

다음 코드를 AI가 예측해 완성

● Ctrl+L 채팅

코드 생성·설명·리팩토링 대화

● Ctrl+K 인라인

선택 코드를 AI가 즉시 수정

● @폴더 분석

프로젝트 전체를 AI가 이해 후 개선 제안

4-6교시

풀스택 앱 빠른 개발 (프론트+백엔드)

⚡ 개발 시간 70% 단축 체험

03

PRD → 프로젝트 구조 자동 생성 (Cursor Composer)

📄 Step 1 — PRD 작성 → Ctrl+I (Composer)에 전달 → 전체 구조 자동 생성

Composer 프롬프트 (Ctrl+I)

다음 PRD를 바탕으로 풀스택 앱을 만들어줘.

[앱명] TaskFlow - 팀 작업 관리 시스템

[기술 스택]

- Frontend: Next.js 14, Tailwind CSS, shadcn/ui
- Backend: Node.js + Express, Prisma ORM
- DB: PostgreSQL
- 인증: NextAuth.js (Google OAuth)

[핵심 기능]

1. 프로젝트/태스크 CRUD
2. 팀 멤버 초대 & 권한 관리
3. 실시간 상태 업데이트 (Socket.io)
4. 대시보드 & 통계

Composer가 자동 생성한 결과

```
taskflow/  
├── frontend/  
│   ├── app/           # Next.js App Router  
│   ├── components/  
│   │   ├── ui/        # shadcn 컴포넌트  
│   │   └── features/   # 기능별 컴포넌트  
│   └── lib/            # 유틸, 혹은  
├── backend/  
│   ├── src/  
│   │   ├── routes/    # API 라우터  
│   │   ├── services/  # 비즈니스 로직  
│   │   └── middleware/  
│   └── prisma/         # 스키마 & 마이그레이션  
└── docker-compose.yml
```

🕒 기존 개발 vs Cursor AI Composer 시간 비교

작업	기존 개발	Cursor Composer	절감
프로젝트 초기 구조 생성	2~4시간	5~10분	95%↓
API 엔드포인트 10개 작성	1~2일	30분	87%↓
DB 스키마 + 마이그레이션	2~3시간	10분	90%↓
프론트 컴포넌트 10개	1~2일	1시간	80%↓
단위 테스트 50개 작성	1일	20분	97%↓

실습 # — 풀스택 앱 빠른 개발

제시
문제

과제 1: 풀스택 앱 빠른 개발 (프론트+백엔드)

PRD → API 설계 → 프론트 연동 동시 생성 | 개발 시간 70% 단축 체험

할루시네이션 방지 & Cursor 검증 전략

⚠ Cursor AI 할루시네이션 유형



존재하지 않는 패키지

import fake-lib — npm install 시 에러 발생



잘못된 API 시그니처

실제와 다른 함수 인자·반환값 사용



틀린 비즈니스 로직

요구사항과 다른 계산식·조건문 생성



DB 스키마 불일치

실제 DB와 다른 컬럼명·타입 참조



보안 취약점 코드

SQL Injection 등 취약한 패턴 생성

🛡 Cursor AI 검증 전략 5단계

1

.cursorrules 구체화

기술 스택·버전·금지 패턴 명시
→ 잘못된 패키지·API 사용 차단

```
# .cursorrules
- axios: 1.6.x (fetch 사용 금지)
- prisma: 5.x 문법만 사용
- any 타입 절대 금지
```

2

@컨텍스트 적극 활용

@파일로 실제 스키마·타입 참조 강제
→ 존재하는 코드 기반으로만 생성

```
# Chat 예시
@prisma/schema.prisma 참고해서
Task 서비스 작성해줘.
```

3

Composer Diff 필수 검토

Accept All 전 반드시 변경 내용 확인
→ 의도치 않은 파일 수정 방지

```
# Composer 사용 후
# 각 파일 Diff를 클릭해서
# 변경 내용 확인 후 Accept
```

4

터미널에서 즉시 실행 검증

생성 직후 빌드·테스트 실행
→ 컴파일 에러·테스트 실패 즉시 발견

```
# Chat에서 지시:
"구현 후 npm run build &&
npm test 실행하고 에러 있으면
수정해줘"
```



항금 규칙: Cursor AI 출력물은 '초안'입니다. Diff 확인 → 실행 → 검토 → 수정의 루프를 항상 지키세요.

Cursor Rules & 커스텀 명령어 자동화

Cursor Rules 고급 설정 (.cursorrules)

.cursorrules - 코드 리뷰 & 기능 생성 규칙

```
# === 코드 리뷰 요청 시 ===
When asked to review code, always check:
1. SOLID 원칙 위반 여부 (각 항목 설명 포함)
2. 보안 취약점 (OWASP Top 10 기준)
3. 성능 개선 포인트 (Big-O 포함)
4. 테스트 커버리지 제안
5. 마크다운 표 형식으로 정리

# === 새 기능 구현 요청 시 ===
When implementing features, always:
1. 기존 패턴 (@src/services 참고)을 따를 것
2. TypeScript strict mode 준수
3. 구현 후 테스트 코드도 함께 생성
```

.cursorrules - 자동화 파이프라인 규칙

```
# === PR 준비 체크리스트 ===
Before finalizing any code, verify:
□ npm run build 통과
□ npm test 통과 (커버리지 80%+)
□ ESLint 에러 없음
□ 모든 함수에 JSDoc 주석
□ README.md 관련 섹션 업데이트

# === 파일 생성 컨벤션 ===
- Service 파일: src/services/*.service.ts
- Route 파일:  src/routes/*.route.ts
- Type 파일:   src/types/*.type.ts
- Test 파일:   src/__tests__/*.test.ts
```

Chat 자동화 & 반복 작업 처리

Chat 반복 작업 자동화 예시

```
# 1. 테스트 없는 파일 일괄 처리
"@Codebase 테스트 파일이 없는 서비스를
찾아서 모두 Jest 테스트 작성해줘"

# 2. JSDoc 일괄 추가
"@src/services 폴더의 모든 public
메서드에 JSDoc 주석 추가해줘"

# 3. 의존성 최신화
"package.json 분석하고 보안 취약점 있는
패키지 최신 버전으로 업데이트해줘"
```

GitHub Actions + Cursor 규칙 연동

```
# .github/workflows/ai-check.yml
- name: Lint Rules Validation
  run: |
    # .cursorrules에 정의한 규칙을 CI에서도 검사
    npm run lint
    npm run typecheck
    npm test -- --coverage --
    coverageThreshold: '{"global":{"lines":80}}'
# PR 머지 전 자동 품질 게이트
```

실습 # — .cursorrules 작성

제시 문제

.cursorrules 핵심 설정

project: MagicSquare · style: PEP8 + 타입힌트 · testing: pytest AAA패턴 · architecture:

ECB(boundary/control/entity) · forbidden: print()디버깅

프로젝트\.cursor\

└rules

.cursorrules

확인

• 파일이 채워지면 바로 아래 프롬프트로 테스트하세요.

실습 # — .cursorrules 확장

제시 문제

- D:\1-3-2..MagicSquare_개선

| .cursorrules

|

└─.cursor

└─rules

magicsquare-ecb-architecture.mdc

magicsquare-forbidden.mdc

magicsquare-project.mdc

magicsquare-python-code-style.mdc

magicsquare-tdd-testing.mdc

7번째 시간

Cursor AI 고급 — AI 에이전트 & API 연동

멀티파일 에이전트 + OpenRouter API + 냉장고 앱 + AI 다이어리



Cursor AI 에이전트 체인 — 자율 실행 파이프라인



Cursor Composer + Chat = AI 에이전트 루프 (Thought → Action → Observation 반복)



사용자 목표
입력



Cursor
분석 (Thought)



Action
(파일 수정·실행)



Observation
(결과 확인)



완료 또는
반복

반복 (목표 달성까지)



실전 에이전트 — PR 자동 리뷰 (Cursor Chat + OpenRouter)

Cursor Chat에서 PR 리뷰 에이전트 실행

1. Chat에서 @terminal과 @git 컨텍스트 활용

"@terminal 'git diff HEAD~1'을 실행해서 변경 파일 목록을 가져오고,
각 파일을 @코드베이스에서 읽어 다음을 분석해줘:

1) 보안 취약점 (OWASP Top 10), 2) 테스트 누락, 3) 성능 이슈

결과를 review_report.md에 저장하고, 치명적 이슈가 있으면 'BLOCK', 없으면 'APPROVE'로 마무리해줘."

Composer가 자율적으로: git diff 실행 → 파일 읽기 → 분석 → 리포트 작성 → 결론 도출



Cursor의 Composer는 @terminal을 통해 실제 명령어를 실행하고 결과를 반영합니다. 에이전트처럼 반복 루프를 돌며 목표를 달성합니다.

Cursor 에이전트 모드 — 멀티파일 동시 수정

Ctrl+Shift+L로 여러 파일을 한 번에 리팩토링

javascript

```
// 에이전트 모드 사용법 (Ctrl+Shift+L)
// 프롬프트: "@web_version 폴더를 분석해서 데스크톱 최적화 확인 후 수정해줘"

// 에이전트가 자동으로 실행하는 과정:
// 1. web_version/ 폴더 전체 스캔
// 2. CSS 미디어쿼리 분석 (모바일 최적화만 발견)
// 3. 데스크톱 레이아웃 수정 계획 수립
// 4. 파일 동시 수정 시작...

// ✅ 수정 완료 목록:
// web_version/index.html - 사이드바 2열 레이아웃 추가
// web_version/style.css - max-width: 1400px + Grid
// web_version/script.js - 데스크톱 단축키 추가

// 다중 에이전트 협업 프롬프트:
// "code-bug-analyzer로 코드 리뷰 후,
// performance-optimizer가 수정해서 최적화하고,
// ux-design-advisor가 사용자 경험을 개선해 줘"
```



코드 리뷰어

버그·컨벤션·보안 취약점 검토



성능 최적화

병목 발견·속도 개선·메모리 최적화



UX 디자이너

화면·버튼·에러 메시지 개선



프로덕트 매니저

PRD 작성·기능 명세·일정 관리

OpenRouter API — 멀티모델 라우팅

🌐 OpenRouter란? — 단일 API로 100+ LLM 모델에 접근 / Cursor AI와 함께 사용

개발자 앱
(단일 API 키)



OpenRouter
라우터

스마트 라우팅 예시 (JS)

```
const client = new OpenAI({ baseURL:"https://openrouter.ai/api/v1",  
  apiKey:process.env.OPENROUTER_API_KEY });  
async function smartChat(prompt, complexity="medium") {  
  const modelMap = { simple:"google/gemini-flash-1.5",  
    medium:"anthropic/claude-3.5-sonnet", complex:"anthropic/claude-  
    opus-4" };  
  return client.chat.completions.create({  
    model:modelMap[complexity], messages:[{role:"user",content:prompt}]  
  });  
}
```



Cursor AI에서 OpenRouter 모델 사용하기

Cursor Settings > Models > Add Custom Model

```
baseURL: https://openrouter.ai/api/v1  
apiKey: OPENROUTER_API_KEY  
model: anthropic/claude-3.5-sonnet (또는 원하는 모델)
```

Claude 3.5
Sonnet

GPT-4o

Gemini 1.5
Pro

Llama 3.1
(오픈소스)



비용 최적화

태스크별 최적 모델
선택
(빠른 작업은 저렴한
모델)



Fallback 처리

모델 장애 시 자동 전
환
99.9% 가용성 보장



사용량 추적

모델별 비용·토큰 사
용량
대시보드에서 통합 관
리



오픈소스 접근

Llama, Mistral 등
무료 모델도 동일 API

OpenRouter API 연동 — 무료 AI 모델 활용

.env 보안 관리 + 무료 모델로 AI 기능 구현

javascript

```
# .env 파일 설정 (절대 Git에 커밋 금지!)
OPENROUTER_API_KEY=sk-or-xxxxxxxxxxxx

# .gitignore에 반드시 추가
.env
*.env.local

---
// OpenRouter API 연동 프롬프트:
> OpenRouter에서 받은 API 키를 .env 파일로 저장했어.
  이 키를 안전하게 사용할 수 있도록 준비해 줘.

> 이제 준비된 API가 실제로 작동하는지 테스트해 줘.
  이미지 인식은 google/gemma-3-27b-it:free 모델,
  텍스트 생성은 deepseek/deepseek-chat-v3.1:free 모델 사용.

// Cursor AI 생성 api.js
import dotenv from 'dotenv';
dotenv.config();
```

OpenRouter API 연동 — 무료 AI 모델 활용

.env 보안 관리 + 무료 모델로 AI 기능 구현

javascript

```
async function callAI(model, messages) {
  const res = await fetch("https://openrouter.ai/api/v1/chat/completions", {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${process.env.OPENROUTER_API_KEY}`,
      "Content-Type": "application/json"
    },
    body: JSON.stringify({ model, messages })
  });
  return res.json();
}
```

 이미지 인식 google/gemma-3-27b-it:free

→ 냉장고 사진 재료 인식

 텍스트 생성 deepseek/deepseek-chat-v3.1:free

→ 레시피 추천.감정 분석

OpenRouter API 연동 — 무료 AI 모델 활용

텍스트 모델

모델	상태	비고
meta-llama/llama-3.3-70b-instruct:free	❌ Rate-limited	일시적, 나중에 재시도 가능
meta-llama/llama-3.2-3b-instruct:free	❌ Rate-limited	일시적
qwen/qwen3-coder:free	❌ Rate-limited	일시적
nvidia/nemotron-3-super-120b-a12b:free	✅ 정상	응답 빠름 (3.5s)
nvidia/nemotron-3-nano-30b-a3b:free	⚠️ 빈 응답	모델 자체 문제

비전(이미지) 모델

모델	상태	비고
google/gemma-3-27b-it:free	❌ Rate-limited	일시적
google/gemma-4-31b-it:free	❌ Rate-limited	일시적
google/gemma-4-26b-a4b-it:free	✅ 정상	강아지 이미지 정확히 인식 (4.1s)
nvidia/nemotron-nano-12b-v2-vl:free	⚠️ 빈 응답	모델 자체 문제

실습- API 활용 & AI Agent

이미지 인식 → 레시피 생성 → 사용자 프로필 저장

프롬프트

OpenRouter API를 이용해서, 냉장고 사진에서 재료를 인식하고 레시피를 추천하는 웹 애플리케이션을 만들고 싶어.

1단계: google/gemma-3-27b-it:free 모델로 이미지 인식

2단계: deepseek/deepseek-chat-v3.1:free 로 레시피 생성

3단계: 사용자 프로필 만들어서 레시피 저장

각 단계를 PRD_step1.md, PRD_step2.md, PRD_step3.md로 나누어 저장해 줘

PRD Step 1

이미지 입력 → Gemma-3으로 재료 인식

PRD_step1.md 저장 → 실행: > PRD_step1.md를 실행해.

PRD Step 2

인식 재료 + DeepSeek으로 레시피 생성

PRD_step2.md 저장 → 실행: > PRD_step2.md를 실행해.

PRD Step 3

사용자 프로필 생성 + 레시피 저장 기능

PRD_step3.md 저장 → 실행: > PRD_step3.md를 실행해.

💡 > 메인 애플리케이션을 실행해서 N단계의 결과를 테스트해 줘 — 각 단계 실행 후 바로 결과 확인

8번째 시간

MCP 입문 — AI를 실행자로 만드는 도구

Notion · Sequential Thinking · Playwright · GitHub · Supabase MCP 연결



MCP(Model Context Protocol) 정의

Model Context Protocol (MCP)

2024년 11월 Anthropic이 발표한 오픈 표준 — AI 모델(LLM)과 외부 데이터 소스·도구 사이의 연결을 표준화하는 개방형 프로토콜

💡 비유: AI를 위한 USB-C 포트



USB-C

어떤 기기도 같은 포트에 연결



MCP

어떤 AI도 같은 방법으로 외부 연결

📌 핵심 사실



2024.11.25 발표

Anthropic이 오픈소스로 공개



오픈 표준

OpenAI·Google DeepMind도 공식 채택



N×M 문제 해결

기존 커스텀 커넥터 방식 대체

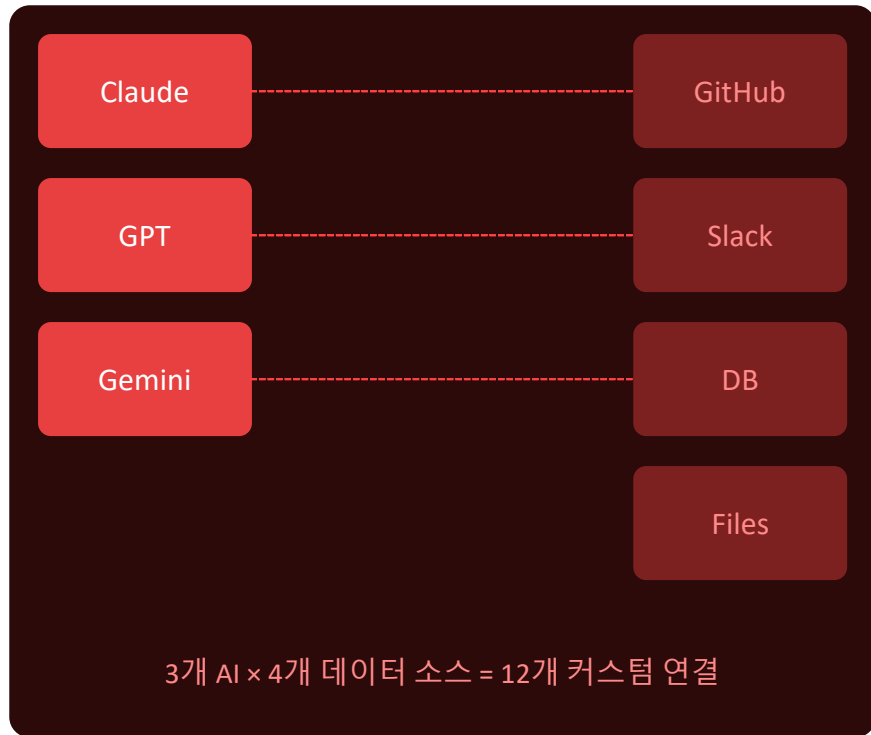


Linux Foundation 이관

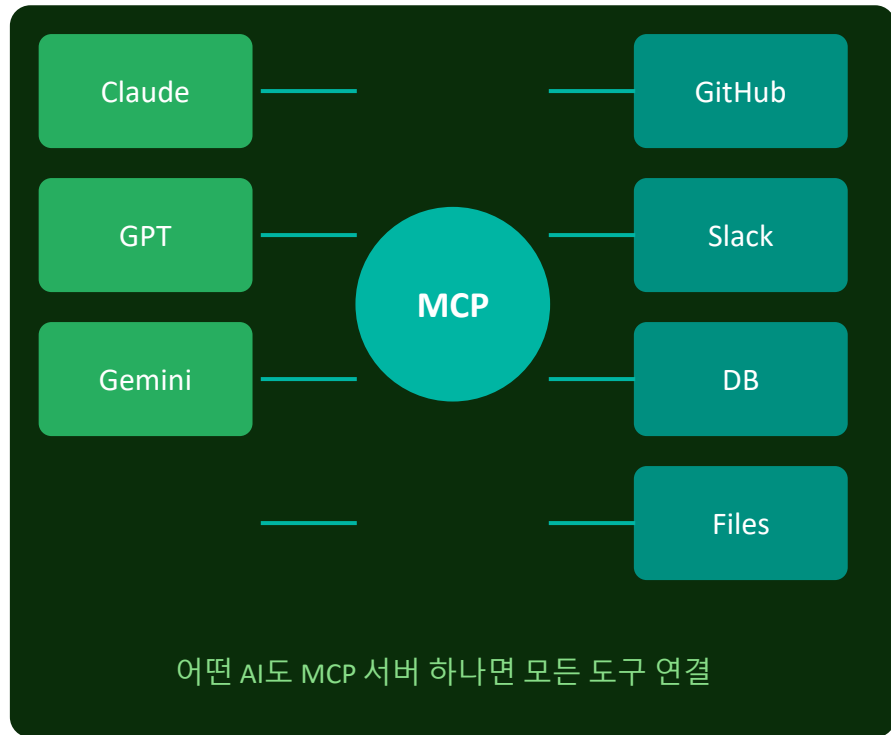
2025.12 AAIF로 중립성 확보

MCP 등장 배경 — N×M 통합 문제

❌ MCP 이전: $N \times M$ 통합 지옥



✅ MCP 이후: 표준 1개로 모두 연결



MCP 아키텍처 — Host · Client · Server



MCP Host

(예: Claude Desktop)

사용자가 직접 사용하는 AI 애플리케이션
여러 MCP Client를 포함·관리

예시

Claude Desktop, Cursor IDE, VS Code

↕ JSON-RPC 통신



MCP Client

(프로토콜 처리)

Host 내부의 프로토콜 처리 담당
MCP Server와 1:1 연결 설정

예시

Host 내 내장 컴포넌트

↕ JSON-RPC 통신



MCP Server

(기능 제공)

특정 도구·데이터 소스 기능 제공
Resources, Tools, Prompts 노출

예시

Filesystem, GitHub, Slack, DB 서버

데이터 흐름: 사용자 요청 → Host → Client → Server → 외부 데이터/도구 → Client → Host → 사용자 응답

MCP 핵심 기능 — Resources · Tools · Prompts



Resources

외부 데이터·파일·DB의 정보를 AI에 제공

- 읽기 전용 정보 조회
- 파일, 데이터베이스, API 응답
- 컨텍스트 제공 (실행 없음)

예시: 파일 내용 읽기, DB 레코드 조회



Tools

실제 동작을 실행하는 함수·명령어

- 계산, 파일 생성·수정·삭제
- API 호출, 외부 서비스 제어
- 사이드 이펙트 발생 가능

예시: 파일 쓰기, 명령어 실행, API 전송



Prompts

AI와 서버 간 재사용 가능한 프롬프트 템플릿

- 표준화된 워크플로우 정의
- 반복 작업 자동화
- LLM-서버 통신 패턴 최적화

예시: 코드 리뷰 요청, 요약 템플릿

MCP 서버 저장소란?

MCP 서버 저장소 = 개발자들이 만든 MCP 서버를 공유·검색·설치할 수 있는 중앙 허브

전통적인 npm·pip 패키지 저장소처럼, AI가 사용할 수 있는 도구(Tools)·데이터(Resources)를 모아놓은 공간입니다.



GitHub (공식 서버)

github.com/modelcontextprotocol/servers

Anthropic이 직접 관리하는 공식 레퍼런스 서버 모음
파일시스템·Git·Brave Search·PostgreSQL 등 포함



Smithery

smithery.ai

5,900개+ MCP 서버 등록된 세계 최대 MCP 마켓플레이스
클릭 몇 번으로 설치·연결 가능, GitHub 계정으로 가입



awesome-mcp-servers

github.com/punkpeye/awesome-mcp-servers

커뮤니티가 엄선한 고품질 MCP 서버 큐레이션 목록
카테고리별 분류 (개발·검색·업무·데이터 등)

사전 구축된 MCP 서버 생태계

MCP Market Map

A work in progress.

Top MCP Clients

Chat apps



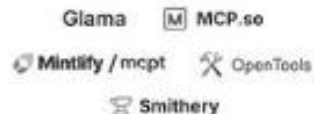
Task Automation



Coding



MCP Marketplace



Top MCP Servers

Database



Scraping & Search



Art & Design



Email



Evaluation



Agent Execution Environments



Debugging



Productivity Tools



Payments



Ticketing



Monitoring & Observability



Server Generation & Curation



Server Hosting



Connection Management



Charts provided herein are for informational purposes only and should not be relied upon when making any investment decision. Past performance is not indicative of future results. Some of the data should be taken as non-final, subject to ongoing user contributions for more information.

Smithery란? — MCP 서버의 앱스토어

Smithery = MCP 서버의 앱스토어 + 중앙 레지스트리 + 개발 플랫폼

5,900개+ 서버 보유 · Claude Desktop·Cursor·VS Code 등 모든 MCP 클라이언트 지원 · GitHub 계정으로 가입



서버 탐색

카테고리·검색으로
원하는 MCP 서버 탐색



원클릭 설치

JSON 복사 또는 CLI
명령어로 즉시 설치



Playground

설치 전 서버 기능을
온라인에서 미리 테스트



서버 배포

내가 만든 서버를
Smithery에 공개 배포



API Key 관리

서버별 인증 키 발급
안전하게 관리



Observability

서버 사용량·오류
모니터링 대시보드

클라이언트 무관(client-agnostic) 설계 — 어떤 MCP 클라이언트에서도 Smithery 서버 사용 가능

Smithery 가입 및 서버 탐색 실습

smithery.ai 접속

①

브라우저에서 <https://smithery.ai> 접속
우측 상단 "Sign in" 클릭

GitHub 계정 연동

②

GitHub 계정으로 로그인
(GitHub 없으면 먼저 가입 필요)

Smithery Key 확인

③

로그인 후 계정 설정에서
Smithery Key 발급 확인 (서버 설치 시 필요)

서버 탐색

④

검색창에 키워드 입력 또는
카테고리(개발/검색/업무) 탐색

서버 탐색 화면 구성

서버 목록 → 클릭 → Overview(설명) · Tools(기능 목록) · API · Playground 탭 확인 → Connect에서 설치 JSON 복사

대표 MCP 서버 ①— Sequential Thinking: 단계적 추론

Sequential Thinking이란?

복잡한 문제를 단계적 사고(step-by-step reasoning)로 분해하여 Claude의 추론 품질을 높이는 MCP 서버
→ 수학·코드 설계·전략 계획 등 복잡한 작업에서 오류를 줄이고 체계적 사고를 강제



수학·계산

복잡한 수식을 단계별로 분해
중간 계산 포함 정확도 향상



시스템 설계

아키텍처 결정을 순서대로
각 단계 근거와 함께 제시



디버깅

버그 원인 추적을
논리적 순서로 분석



전략 계획

복잡한 프로젝트 계획을
의존성 순서로 정리

대표 MCP 서버 ② — Context7: 최신 라이브러리 문서 제공

Context7이란?

프로그래밍 라이브러리의 최신 공식 문서와 코드 예제를 실시간으로 AI에 제공하는 MCP 서버
→ Claude가 최신 React·FastMCP·Next.js 등의 정확한 API를 코딩 시 직접 참조

✗ Context7 없이

- Claude가 학습 당시의 오래된 버전 API 사용
- 존재하지 않는 메서드 Hallucination
- 버전 충돌 오류 코드 생성
- "이 코드 안 돼요" 반복 디버깅

✓ Context7 사용 시

- 프롬프트에 "use context7" 추가
- 최신 공식 문서 기반 코드 생성
- 실제 동작하는 정확한 예제 제공
- 라이브러리 변경사항 즉시 반영

완전한 AI 개발 워크플로우 — 처음부터 배포까지

AI 도구들을 조합해 실무 수준의 개발 파이프라인 구성



✅ 이 7단계 워크플로우 = 현업 스타트업 개발 속도 · 혼자서 팀 역할 수행 가능

💡 Gemini CLI (무료) = 초기 구조 생성 | Cursor AI (\$20) = 세부 품질 향상 | MCP = 자동화·배포

C++ & Java 개발자 최적 AI 워크플로우

언어별로 최적화된 AI 도구 조합 전략



C++17

추천 도구 조합

- ▶ Gemini CLI: CMakeLists.txt + 헤더 구조
- ▶ Cursor AI: 클래스 구현 + 스마트 포인터
- ▶ .cursorrules: RAll, const correctness 강제
- ▶ Context7 MCP: C++20 표준 최신 참조
- ▶ Google Test: Ctrl+L 자동 생성
- ▶ ASAN: 메모리 누수 검사 자동화



Java 17+

추천 도구 조합

- ▶ Gemini CLI: pom.xml + 프로젝트 구조 생성
- ▶ Cursor AI: Controller·Service·Repository
- ▶ .cursorrules: Record, Optional, Stream 강제
- ▶ Context7 MCP: Spring Boot 최신 API 참조
- ▶ Playwright: REST API 엔드포인트 자동 테스트
- ▶ JUnit 5: Ctrl+L '테스트 만들어줘' 자동 생성

✓ 공통 원칙: PRD 먼저 → Gemini CLI로 뼈대 → Cursor AI로 살 채우기 → MCP로 자동화·배포

T H A N K Y O U

바이트코딩으로 세상을 바꾸세요

Gemini CLI × Cursor AI × MCP × Java × C++ × VSCode
